NEXT ENGINEER · _SHEERAT_

Operating System

BCS401

One-day revision notes: definitions, solved examples, code and important questions

Program: B.Tech | **Branches:** CSE, IT, CSE-AIML, CSE-DS, AIDS | **Semester:** 4 | **Credits:** 4

L-T-P: 3-1-0 | **Scheme:** AKTU B.Tech 2nd year, 2023-24 onwards

UNIT 1 Introduction and OS Structure

UNIT 2 Concurrent Processes and Synchronization

UNIT 3 CPU Scheduling and Deadlock

UNIT 4 Memory Management

UNIT 5 I/O, Disk Scheduling and File Systems

Original revision notes written for the AKTU syllabus. Every unit has the same parts:

1. **2-mark definitions**: short answers you can write word for word in Section A.
2. **Core concepts**: comparison tables and algorithms.
3. **Solved examples**: the numericals that always appear (scheduling, Banker's, page replacement, disk scheduling, memory allocation).
4. **Code to remember**: synchronization pseudocode examiners ask you to write.
5. **Important questions**: the long questions to prepare, with what your answer must contain.

How to use these notes the day before the paper

- **Morning (3 hours):** Units 1 and 2. Learn OS types and kernel structures, then Peterson's solution, semaphores and the classical problems.
- **Afternoon (3 hours):** Unit 3: solve the scheduling example with all four algorithms and the Banker's algorithm example by hand. These are guaranteed marks.
- **Evening (2 to 3 hours):** Units 4 and 5: page replacement (FIFO, LRU, Optimal), allocation (first/best/worst fit), disk scheduling (FCFS, SSTF, SCAN, C-SCAN, LOOK), then the "Last-hour checklist".
- **In the exam:** always draw the Gantt chart, then a table of CT, TAT and WT, then averages. For page replacement draw the frame table with fault markers. For disk scheduling, draw the head movement line and add up distances.

AKTU paper pattern (theory, 70 marks, 3 hours): Section A has 7 short questions of 2 marks, all compulsory (14). Section B has 5 questions of 7 marks, attempt any 3 (21). Section C has 5 questions (one per unit), each with two parts, attempt one part of each, 7 marks each (35). Every unit gets its own Section C question, so do not skip any unit.

UNIT 1 Introduction and OS Structure

✍️ 2-mark definitions

- **Operating system:** system software that manages computer hardware and provides services to programs; it acts as an intermediary between the user and the hardware, and as a **resource manager**.
- **Functions of an OS:** process management, memory management, file management, I/O/device management, storage management, protection and security, user interface, networking.
- **Batch system:** jobs with similar needs are grouped and run one after another without user interaction.
- **Multiprogramming:** several programs are kept in memory so the CPU always has one to run, improving CPU utilization.
- **Time-sharing (multitasking):** CPU time is divided into small slices among many users/programs so each gets fast response.
- **Real-time system:** must respond within strict time limits. **Hard** real-time (deadline must never be missed, e.g. airbag controller) and **soft** real-time (occasional misses tolerated, e.g. video streaming).
- **Multiprocessor system:** two or more CPUs sharing memory and devices; *symmetric* (all CPUs equal) or *asymmetric* (master–slave).
- **Multithreaded system:** a process can have several threads that run concurrently and share its resources.
- **System call:** the programming interface through which a program requests an OS service (e.g. `fork()`, `read()`, `open()`).
- **Kernel:** the core part of the OS that runs in privileged (kernel) mode.
- **Reentrant kernel:** a kernel whose code can be executed by several processes at the same time safely, because it does not modify shared global data without protection; each process keeps its own data on its own stack.
- **Dual mode:** user mode and kernel mode, switched by a mode bit, to protect the OS from user programs.

💡 Core concepts

Types of operating systems

Type	Key idea	Example
Batch	Jobs grouped and run without interaction	Early IBM mainframes
Multiprogrammed	Many jobs in memory; CPU switches on I/O wait	
Time-sharing	Short time slices; interactive	UNIX
Real-time	Strict deadlines	VxWorks, RTOS in cars

Type	Key idea	Example
Multiprocessor	Several CPUs	Modern servers
Distributed	Many computers appear as one system	
Multiuser	Many users at once	Linux servers
Multithreaded	Threads inside processes	Windows, Linux

OS structures

Structure	Idea	Pros / cons
Simple / monolithic	Whole OS is one big kernel program	Fast; hard to maintain, one bug can crash all
Layered	OS split into layers; layer n uses only layer n−1	Easy to debug; slower, hard to define layers
Microkernel	Minimal kernel (IPC, scheduling, basic memory); services run in user space	Reliable, extensible; slower due to message passing
Modular	Core kernel + loadable modules	Flexible (Linux)
Hybrid	Mix of monolithic and microkernel	Windows, macOS

Monolithic vs microkernel

Monolithic	Microkernel
All services in kernel space	Only essential services in kernel
Fast (function calls)	Slower (message passing)
Large kernel	Small kernel
A failing service may crash the system	Failing service can be restarted
Linux, early UNIX	Mach, QNX, MINIX

System components: process manager, main-memory manager, file manager, I/O system manager, secondary-storage manager, networking, protection system, command interpreter (shell).

OS services: user interface (CLI/GUI), program execution, I/O operations, file-system manipulation, communication, error detection, resource allocation, accounting, protection and security.

🧮 Solved examples

Example 1 (choose the OS type). An aircraft autopilot that must react within 5 ms → **hard real-time OS**. A university server where hundreds of students log in at once → **time-sharing, multiuser OS**.

Example 2 (why dual mode?). If a user program could execute `halt` or change the interrupt table, it could crash the whole system. Such instructions are *privileged*: they run only in kernel mode, reached through a system call (trap).

Key points to remember

```
OS = resource manager + intermediary
Batch → multiprogramming → time-sharing → real-time (hard/soft)
Monolithic (fast, big) vs microkernel (small, reliable, slower)
Layered: layer n uses only n-1
Reentrant kernel: many processes execute kernel code at once safely
User mode ↔ kernel mode via system calls / traps
```

⭐ Important questions

1. Define an operating system and explain its functions and services.
2. Classify operating systems: batch, multiprogramming, time-sharing, real-time, multiprocessor, multiuser, multithreaded.
3. Explain OS structures: layered, monolithic and microkernel. Compare monolithic and microkernel.
4. What is a reentrant kernel? Explain with an example.
5. Explain system components and system calls.

NEXT ENGINEER _sheerat_

UNIT 2 Concurrent Processes and Synchronization

✍️ 2-mark definitions

- **Process:** a program in execution, with its code, data, stack, heap, program counter and registers.
- **Concurrency:** several processes making progress in overlapping time periods (interleaved on one CPU or parallel on many).
- **Race condition:** when the result depends on the order in which processes access shared data.
- **Critical section:** the part of a program where shared data is accessed.
- **Requirements of a critical-section solution: mutual exclusion** (only one process in the CS), **progress** (if no one is in the CS, the choice of who enters next cannot be postponed forever), **bounded waiting** (a limit on how many times others can enter before a waiting process gets its turn).
- **Semaphore:** an integer variable accessed only through two atomic operations, **wait (P)** and **signal (V)**. **Counting** semaphore (any value) and **binary** semaphore (0/1, like a mutex).
- **Test-and-set:** an atomic hardware instruction that returns the old value of a lock and sets it to true in one step.
- **Busy waiting (spinlock):** a process loops while waiting for a lock, wasting CPU time.
- **Inter-process communication (IPC):** mechanisms that let processes exchange data: **shared memory** and **message passing**.
- **Process creation:** a parent creates a child process (e.g. `fork()` in UNIX), forming a process tree.

💡 Core concepts

Producer–consumer (bounded buffer) problem: a producer adds items to a buffer of size n and a consumer removes them. The producer must wait when the buffer is full; the consumer must wait when it is empty; access to the buffer must be mutually exclusive.

Software solutions for two processes

- **Dekker's algorithm:** the first correct software solution; uses `flag[2]` (wants to enter) and `turn` (whose turn it is to insist).
- **Peterson's solution:** simpler; uses `flag[2]` and `turn`. It satisfies mutual exclusion, progress and bounded waiting.

Hardware solution: test-and-set (or compare-and-swap) gives atomic locking; simple but causes busy waiting and may not ensure bounded waiting without extra code.

Classical synchronization problems

- **Dining philosophers:** 5 philosophers, 5 forks; each needs both neighbouring forks to eat. Naive semaphore solution can **deadlock** (everyone picks up the left fork). Fixes: allow at most 4 to sit, pick up both forks only if both are free (monitor), or odd philosophers pick left first and even pick right first.
- **Sleeping barber:** one barber, one barber chair, n waiting chairs. If no customer, the barber sleeps; an arriving customer wakes the barber, waits if chairs are free, or leaves if all are full. Solved with

semaphores `customers`, `barber`, `mutex` and a counter `waiting`.

- **Readers–writers:** many readers may read together, but a writer needs exclusive access.

IPC models

Shared memory	Message passing
Processes share a memory region	Processes send/receive messages via the kernel
Fast (no kernel after setup)	Slower (system call per message)
Needs synchronization by programmer	Easier for distributed systems
Large data	Small data

Message passing schemes: direct vs indirect (mailboxes), blocking (synchronous) vs non-blocking (asynchronous), buffering (zero, bounded, unbounded capacity).

🧮 Solved examples

Example 1 (race condition). Two processes run `count++` on shared count = 5. Each `count++` is really: load register ← count; register++; count ← register. If both load 5 before either stores, both store 6, so the final value is **6 instead of 7**. A critical section with mutual exclusion prevents this.

Example 2 (semaphore values). A counting semaphore S = 3 (three printers). Five processes do wait(S): the first three get printers (S becomes 0), the next two block (S = −2 in the implementation that counts waiters). When one process signals, one blocked process is woken.

NEXT ENGINEER _sheerat_

⌨️ Code to remember

```
/* Peterson's solution for process i (j = 1 - i) */
flag[i] = true;
turn = j;
while (flag[j] && turn == j);   /* busy wait */
    /* critical section */
flag[i] = false;
    /* remainder section */

/* Semaphore operations (blocking version) */
wait(S):   S.value--;  if (S.value < 0) { add this process to S.list; block(); }
signal(S): S.value++;  if (S.value <= 0) { remove a process P from S.list; wakeup(P); }

/* Producer-consumer with semaphores: mutex = 1, empty = n, full = 0 */
Producer:                          Consumer:
  while (true) {                     while (true) {
    produce item;                      wait(full);
    wait(empty);                       wait(mutex);
    wait(mutex);                       remove item from buffer;
    add item to buffer;                signal(mutex);
    signal(mutex);                     signal(empty);
    signal(full);                      consume item;
  }                                  }

/* Test-and-set lock */
boolean TestAndSet(boolean *target) { boolean rv = *target; *target = true; return rv; }
while (TestAndSet(&lock));   /* critical section */   lock = false;
```

⭐ Important questions

1. What is the critical section problem? State its requirements and explain Peterson's (or Dekker's) solution.
2. What are semaphores? Explain wait and signal, and solve the producer–consumer problem using semaphores.
3. Explain the dining philosophers problem and its solution; explain how deadlock can occur.
4. Explain the sleeping barber problem with a semaphore solution.
5. Explain inter-process communication: shared memory and message passing.
6. Explain the test-and-set instruction and how it achieves mutual exclusion.

UNIT 3 CPU Scheduling and Deadlock

✍️ 2-mark definitions

- **Process states:** new, ready, running, waiting (blocked), terminated.
- **Process Control Block (PCB):** the data structure the OS keeps for each process: process ID, state, program counter, registers, scheduling info (priority), memory info (page tables), accounting info, I/O status.
- **Schedulers: long-term** (job scheduler: which jobs enter memory; controls degree of multiprogramming), **short-term** (CPU scheduler: which ready process runs next), **medium-term** (swaps processes out and in).
- **Context switch:** saving the state of the running process and loading the state of the next one; pure overhead.
- **Dispatcher:** gives control of the CPU to the process chosen by the short-term scheduler.
- **Thread:** the smallest unit of CPU execution within a process; threads share code, data and files but have their own stack, registers and PC. **User-level** threads are managed by a library; **kernel-level** threads by the OS.
- **Preemptive vs non-preemptive scheduling:** in preemptive scheduling the OS can take the CPU away from a running process; in non-preemptive scheduling a process keeps the CPU until it finishes or blocks.
- **Scheduling criteria:** CPU utilization, throughput, turnaround time, waiting time, response time.
- **Formulas: Turnaround time (TAT) = Completion time − Arrival time; Waiting time (WT) = TAT − Burst time; Response time = first run − arrival.**
- **Deadlock:** a set of processes each waiting for a resource held by another in the set, so none can proceed.
- **Starvation:** a process waits indefinitely because others keep getting the resource; **aging** (raising priority over time) prevents it.

💡 Core concepts

Process transition diagram: new → (admitted) → ready → (dispatch) → running → (exit) → terminated; running → (interrupt) → ready; running → (I/O or event wait) → waiting → (I/O done) → ready.

Scheduling algorithms

Algorithm	Preemptive?	Notes
FCFS	No	Simple; **convoy effect** (short jobs wait behind long ones)
SJF	No	Minimum average waiting time; needs burst-time prediction; starvation possible
SRTF (preemptive SJF)	Yes	Switches when a shorter job arrives
Priority	Either	Starvation → fix with aging

Algorithm	Preemptive?	Notes
Round Robin	Yes	Time quantum q; good response time; large q → FCFS, tiny q → too many context switches
Multilevel queue	Either	Separate queues (system, interactive, batch) with fixed priority
Multilevel feedback queue	Yes	Processes move between queues based on behaviour

Multiprocessor scheduling: asymmetric (one master CPU schedules) or symmetric (each CPU schedules itself); issues of **processor affinity** and **load balancing** (push/pull migration).

Four necessary conditions for deadlock (Coffman): mutual exclusion, **hold and wait**, **no preemption**, **circular wait**. All four must hold at once.

Handling deadlock

Method	How
Prevention	Make sure one of the four conditions can never hold (e.g. request all resources at once; impose an order on resources to break circular wait)
Avoidance	Grant a request only if the system stays in a **safe state** (Banker's algorithm)
Detection and recovery	Let deadlock happen, detect it (wait-for graph / detection algorithm), then recover by killing processes or preempting resources
Ignore (ostrich)	Assume it is rare (used by most desktop OSs)

Resource allocation graph: if every resource has one instance, a **cycle means deadlock**; with multiple instances, a cycle means deadlock is *possible*.

Banker's algorithm: Need = Max − Allocation. **Safety algorithm:** Work = Available; repeatedly find a process with Need ≤ Work, pretend it finishes (Work += its Allocation). If all processes can finish, the state is safe and that order is a **safe sequence**. **Resource-request algorithm:** if Request ≤ Need and Request ≤ Available, pretend to allocate and run the safety algorithm; grant only if still safe.

🧮 Solved examples

Processes for Examples 1 to 4:

Process	Arrival	Burst
P1	0	5
P2	1	3
P3	2	1
P4	3	2

Example 1 (FCFS). Gantt: | P1 0–5 | P2 5–8 | P3 8–9 | P4 9–11 |

Process	CT	TAT	WT
P1	5	5	0
P2	8	7	4
P3	9	7	6
P4	11	8	6

Average TAT = 27/4 = **6.75**, average WT = 16/4 = **4.0**.

Example 2 (SJF, non-preemptive). Gantt: | P1 0–5 | P3 5–6 | P4 6–8 | P2 8–11 |

Process	CT	TAT	WT
P1	5	5	0
P2	11	10	7
P3	6	4	3
P4	8	5	3

Average TAT = 24/4 = **6.0**, average WT = 13/4 = **3.25**.

Example 3 (SRTF, preemptive). Gantt: | P1 0–1 | P2 1–2 | P3 2–3 | P2 3–5 | P4 5–7 | P1 7–11 | (At t = 3, P2 and P4 both have 2 units left; the tie goes to P2, which arrived first.)

Process	CT	TAT	WT
P1	11	11	6
P2	5	4	1
P3	3	1	0
P4	7	4	2

Average TAT = 20/4 = **5.0**, average WT = 9/4 = **2.25** (the best of the four).

Example 4 (Round Robin, q = 2). Newly arrived processes join the ready queue before a preempted one. Gantt: | P1 0–2 | P2 2–4 | P3 4–5 | P1 5–7 | P4 7–9 | P2 9–10 | P1 10–11 |

Process	CT	TAT	WT
P1	11	11	6
P2	10	9	6
P3	5	3	2
P4	9	6	4

Average TAT = 29/4 = **7.25**, average WT = 18/4 = **4.5**.

Example 5 (Banker's algorithm). Resources A = 10, B = 5, C = 7. Available = (3, 3, 2).

Process	Allocation	Max	Need
P0	0 1 0	7 5 3	7 4 3
P1	2 0 0	3 2 2	1 2 2
P2	3 0 2	9 0 2	6 0 0
P3	2 1 1	2 2 2	0 1 1
P4	0 0 2	4 3 3	4 3 1

- Work = (3,3,2). P1 Need (1,2,2) ≤ Work → Work = (5,3,2).
- P3 (0,1,1) ≤ Work → Work = (7,4,3).
- P4 (4,3,1) ≤ Work → Work = (7,4,5).
- P0 (7,4,3) ≤ Work → Work = (7,5,5).
- P2 (6,0,0) ≤ Work → Work = (10,5,7).
- **Safe sequence: ⟨P1, P3, P4, P0, P2⟩**, so the system is in a safe state.

⌨️ Code to remember

```
Banker's safety algorithm
  Work = Available;  Finish[i] = false for all i
  repeat
     find i with Finish[i] = false and Need[i] ≤ Work
     Work = Work + Allocation[i];  Finish[i] = true;  add Pi to the safe sequence
  until no such i
  safe  ⇔  Finish[i] = true for all i
```

⭐ Important questions

1. Explain the process states with a transition diagram, and the PCB.
2. Explain long-, short- and medium-term schedulers and the scheduling criteria.
3. Solve a scheduling problem with FCFS, SJF, SRTF, priority and Round Robin; find average TAT and WT. *Gantt chart + table for each.*
4. What are threads? Compare user-level and kernel-level threads, and threads vs processes.
5. State the necessary conditions for deadlock; explain prevention, avoidance and detection/recovery.
6. Apply the Banker's algorithm to find a safe sequence and decide whether a request can be granted.

UNIT 4 Memory Management

✍️ 2-mark definitions

- **Bare machine:** hardware with no OS; the user controls the whole machine. **Resident monitor:** a small program always in memory that loads and runs jobs one by one (early batch OS).
- **Logical vs physical address:** the CPU generates logical (virtual) addresses; the **MMU** maps them to physical addresses in RAM.
- **Fixed partitioning (MFT):** memory is divided into fixed-size partitions; each holds one process. Causes **internal fragmentation**.
- **Variable partitioning (MVT):** partitions are created to fit each process exactly. Causes **external fragmentation**; fixed by **compaction**.
- **Internal fragmentation:** wasted space inside an allocated block. **External fragmentation:** enough total free memory exists but it is not contiguous.
- **Paging:** physical memory is divided into fixed-size **frames**, logical memory into **pages** of the same size; a **page table** maps pages to frames. No external fragmentation.
- **Segmentation:** logical memory is divided into variable-size **segments** (code, data, stack) matching the program's logical structure; a segment table holds base and limit.
- **Paged segmentation:** segments are themselves divided into pages, combining the advantages of both.
- **TLB:** translation look-aside buffer, a small fast associative cache of recent page-table entries.
- **Virtual memory:** lets a process run even if only part of it is in RAM, giving each process a large address space.
- **Demand paging:** a page is loaded only when it is first referenced; a missing page causes a **page fault**.
- **Thrashing:** the system spends more time swapping pages than executing, because processes do not have enough frames for their working sets.
- **Locality of reference:** programs access a small set of nearby addresses repeatedly (temporal and spatial locality).

💡 Core concepts

Memory protection schemes: base and limit registers (every address checked: base ≤ address < base + limit), protection bits in page-table entries (read/write/execute), valid–invalid bit.

Partition allocation strategies

Strategy	Choose	Note
First fit	First hole that is big enough	Fast
Best fit	Smallest hole that is big enough	Leaves tiny useless holes
Worst fit	Largest hole	Leaves big reusable holes

Strategy	Choose	Note
Next fit	Like first fit but continues from the last position	

Paging address translation: logical address = (page number p, offset d). If page size = 2^n, the low n bits are the offset. Physical address = frame × page size + offset.

Effective access time (EAT)

- With TLB: EAT = h(t + m) + (1 − h)(t + 2m), where t = TLB time, m = memory time, h = TLB hit ratio.
- With page faults: EAT = (1 − p) × m + p × (page-fault service time).

Paging vs segmentation

Paging	Segmentation
Fixed-size pages	Variable-size segments
Invisible to programmer	Visible (code, data, stack)
Internal fragmentation	External fragmentation
Page table: frame numbers	Segment table: base + limit

Page replacement algorithms: FIFO (replace oldest; suffers **Belady's anomaly**: more frames can give more faults), **Optimal** (replace the page not used for the longest future time; lowest faults, used as a benchmark), **LRU** (replace least recently used; good approximation of optimal), LRU approximations (second chance/clock), LFU, MFU.

Thrashing: cause: too high degree of multiprogramming, too few frames per process. Remedies: working-set model, page-fault-frequency control, reduce multiprogramming.

Cache memory organization: small fast memory holding recently used blocks; works because of locality of reference; mapping (direct, associative, set-associative).

🧮 Solved examples

Example 1 (allocation). Holes: 100, 500, 200, 300, 600 KB. Processes: 212, 417, 112, 426 KB (in order).

Strategy	212 KB	417 KB	112 KB	426 KB
First fit	500 (288 left)	600 (183 left)	288-hole (176 left)	**must wait**
Best fit	300	500	200	600
Worst fit	600 (388 left)	500 (83 left)	388-hole (276 left)	**must wait**

Best fit places all four here.

Example 2 (page replacement, 3 frames). Reference string: 7 0 1 2 0 3 0 4 2 3 0 3 2.

Algorithm	Page faults	Hits
FIFO	**10**	3
LRU	**9**	4

Algorithm	Page faults	Hits
Optimal	**7**	6

Optimal trace: 7, 0, 1 (3 faults) → 2 replaces 7 → 0 hit → 3 replaces 1 → 0 hit → 4 replaces 0 (frames 2, 4, 3) → 2 hit → 3 hit → 0 replaces 4 → 3 hit → 2 hit. Total **7 faults**.

Example 3 (address translation). Page size = 4 KB = 4096. Logical address 13000: page = 13000 div 4096 = **3**, offset = 13000 − 12288 = **712**. If page 3 is in frame 7, physical address = 7 × 4096 + 712 = **29384**.

Example 4 (EAT with TLB). TLB = 20 ns, memory = 100 ns, hit ratio = 80%. EAT = 0.8(20 + 100) + 0.2(20 + 200) = 96 + 44 = **140 ns**.

Example 5 (EAT with page faults). Memory access = 200 ns, fault service = 8 ms, fault rate p = 0.001. EAT = 0.999 × 200 + 0.001 × 8,000,000 ≈ 199.8 + 8000 = **8199.8 ns** (about 40 times slower). This is why the fault rate must be tiny.

Formulas to remember

```
Physical address = frame × page size + offset
EAT (TLB) = h(t + m) + (1 - h)(t + 2m)
EAT (faults) = (1 - p)·m + p·(fault service time)
Pages in logical space = 2^(logical bits) / page size
Belady's anomaly: only FIFO (not LRU/Optimal, which are stack algorithms)
```

⭐ Important questions

1. Explain multiprogramming with fixed and variable partitions; internal vs external fragmentation; compaction.
2. Solve first/best/worst fit allocation problems.
3. Explain paging with a diagram (page table, TLB), and compute EAT.
4. Explain segmentation and paged segmentation; compare paging and segmentation.
5. What is virtual memory and demand paging? Find the performance of demand paging.
6. Solve page replacement problems with FIFO, LRU and Optimal; explain Belady's anomaly.
7. What is thrashing? Explain its causes and remedies; explain locality of reference.

UNIT 5 I/O, Disk Scheduling and File Systems

✍️ 2-mark definitions

- **I/O subsystem:** the part of the kernel that manages devices: scheduling, buffering, caching, spooling, error handling and device drivers.
- **Buffering:** holding data temporarily in memory while it moves between devices or between a device and an application. Types: single, double, circular buffering.
- **Spooling:** holding output for a device (like a printer) on disk so several jobs can share it.
- **Seek time:** time to move the disk arm to the required track. **Rotational latency:** time for the sector to rotate under the head. **Transfer time:** time to read/write the data.
- **Disk scheduling:** choosing the order in which pending disk requests are served to reduce total head movement.
- **RAID:** Redundant Array of Independent Disks: using several disks together for performance and/or reliability.
- **File:** a named collection of related information stored on secondary storage. Attributes: name, type, location, size, protection, timestamps.
- **Access methods:** sequential, direct (random), indexed.
- **Directory:** a structure that organizes files: single-level, two-level, tree, acyclic graph, general graph.
- **Allocation methods:** contiguous, linked, indexed.

💡 Core concepts

Disk scheduling algorithms

Algorithm	Idea	Note
FCFS	Serve in arrival order	Fair, often long movement
SSTF	Closest request next	Less movement; can starve far requests
SCAN (elevator)	Move to one end serving requests, then reverse	Goes all the way to the disk end
C-SCAN	Serve in one direction; jump back to the start	More uniform waiting time
LOOK / C-LOOK	Like SCAN / C-SCAN but turn at the last request, not the disk end	Less movement

RAID levels

Level	Technique	Benefit
0	Striping, no redundancy	Speed only

Level	Technique	Benefit
1	Mirroring	Full redundancy
2	Bit-level striping with Hamming code	Error correction (rare)
3	Byte-level striping, dedicated parity disk	
4	Block-level striping, dedicated parity disk	
5	Block-level striping, distributed parity	Common; survives one disk failure
6	Like 5 with two parity blocks	Survives two failures
10 (1+0)	Mirroring + striping	Speed and redundancy

File allocation methods

Method	How	Pros / cons
Contiguous	File in consecutive blocks	Fast, direct access; external fragmentation, hard to grow
Linked	Each block points to the next (e.g. FAT)	No external fragmentation; slow direct access, pointer overhead
Indexed	An index block holds all block addresses (e.g. inode)	Direct access, no fragmentation; index overhead

Free-space management: bit vector, linked free list, grouping, counting.

File sharing and protection: sharing through links (hard and symbolic), access lists, UNIX permissions (owner/group/others × read/write/execute), passwords, encryption. **Implementation issues:** directory implementation (linear list vs hash table), consistency checking, backup and recovery, journaling.

🧮 Solved examples

Example 1 (disk scheduling). Disk has tracks 0–199. Queue: 98, 183, 37, 122, 14, 124, 65, 67. Head at 53.

Algorithm	Order served	Total head movement
FCFS	98, 183, 37, 122, 14, 124, 65, 67	**640**
SSTF	65, 67, 37, 14, 98, 122, 124, 183	**236**
SCAN (toward 0)	37, 14, (0), 65, 67, 98, 122, 124, 183	53 + 183 = **236**
C-SCAN (toward 199)	65, 67, 98, 122, 124, 183, (199), (0), 14, 37	146 + 199 + 37 = **382**
LOOK (toward 199)	65, 67, 98, 122, 124, 183, 37, 14	130 + 169 = **299**
C-LOOK (toward 199)	65, 67, 98, 122, 124, 183, 14, 37	130 + 169 + 23 = **322**

(Some textbooks do not count the C-SCAN return jump; state your assumption in the exam.)

Example 2 (UNIX permissions). `rwxr-x---` means the owner can read, write, execute; the group can read and execute; others have no access (octal **750**).

Key points to remember

```
Disk time = seek + rotational latency + transfer
FCFS, SSTF (starvation), SCAN, C-SCAN, LOOK, C-LOOK
RAID 0 striping, 1 mirroring, 5 distributed parity, 6 double parity, 10 mirror+stripe
Allocation: contiguous, linked (FAT), indexed (inode)
Directories: single-level, two-level, tree, acyclic graph, general graph
```

⭐ Important questions

1. Explain the I/O subsystem, I/O buffering and spooling.
2. Solve disk scheduling problems with FCFS, SSTF, SCAN, C-SCAN and LOOK. *Draw head-movement diagrams.*
3. What is RAID? Explain its levels.
4. Explain the file concept, file organization and access methods.
5. Explain directory structures and file sharing.
6. Explain file allocation methods and free-space management; discuss file-system protection and security.

Last-hour checklist

Read only this in the last hour before the paper.

- ☐ OS types; monolithic vs microkernel; reentrant kernel.
- ☐ Critical section requirements; Peterson's code; semaphore wait/signal.
- ☐ Producer–consumer with semaphores; dining philosophers; sleeping barber.
- ☐ Process states diagram; PCB; three schedulers.
- ☐ TAT = CT − AT; WT = TAT − BT; Gantt chart method.
- ☐ FCFS / SJF / SRTF / RR / priority; convoy effect; aging.
- ☐ Four deadlock conditions; prevention vs avoidance vs detection.
- ☐ Banker's: Need = Max − Allocation; safe sequence.
- ☐ First/best/worst fit; internal vs external fragmentation.
- ☐ Paging translation; EAT formulas; paging vs segmentation.
- ☐ FIFO / LRU / Optimal; Belady's anomaly; thrashing.
- ☐ Disk scheduling totals; RAID levels; file allocation methods.

Books prescribed by AKTU

These are the books named in the official AKTU syllabus. Buy or borrow them through a bookshop or your college library. The WorldCat links show which libraries hold a copy.

#	Title	Author(s)	Publisher	Where to get it legally
1	Operating System Concepts	Abraham Silberschatz, Peter B. Galvin, Greg Gagne	Wiley	WorldCat
2	Operating Systems	Sibsankar Haldar, Alex A. Aravind	Pearson Education	WorldCat
3	An Introduction to Operating Systems	Harvey M. Deitel	Pearson Education	WorldCat
4	Operating Systems: A Concept-Based Approach (2nd ed.)	D. M. Dhamdhere	Tata McGraw-Hill	WorldCat
5	Operating Systems: Internals and Design Principles (6th ed.)	William Stallings	Pearson Education	WorldCat

The free, legal textbook *Operating Systems: Three Easy Pieces* by Remzi and Andrea Arpaci-Dusseau (ostep.org) is a good companion for Units 2 to 5.

Official syllabus: AKTU B.Tech 2nd year CSE syllabus, 2023-24 scheme (PDF)

NEXT ENGI _sheera